

魔兽作业 Debug Tool

1. 程序功能介绍

本项目是一个面向“魔兽世界终极版”作业的调试辅助工具。该作业输出量很大，一组测试数据可能产生数万行文本，如果只靠人工逐行比对，很难快速判断错误最早从哪里出现。因此本工具的核心目标是：自动运行或读取学生程序输出，与标准输出 `data.out` 逐行比较，定位第一处差异，并把差异发生前 1 小时内的相关战场事件筛选出来，帮助使用者快速回溯 bug。

程序当前提供命令行版和 Qt 图形界面版两种使用方式。

命令行版 `debug_tool.cpp` 支持三类输入：

1. 直接传入已经编译好的可执行文件，工具使用 `data.in` 运行程序并生成实际输出。
2. 直接传入 `.cpp` 源文件，工具自动调用 `clang++ -std=c++17 -O2` 编译，再运行并比对。
3. 使用 `--compare-output` 模式，直接分析已有输出文件，例如 `my.out`。

定位到第一处差异后，命令行版会输出差异所在的 `data.out` 行号、Case 编号、Case 内部行号、时间戳、Expected/Actual 内容、关注城市窗口、bug 前 1 小时的相关事件，以及错误前最近一次的红蓝司令部生命元状态。

1.1 命令行用法

命令行版的通用参数格式如下：

```
1 debug_tool <program_or_cpp> [input_file] [expected_output] [actual_output]
2 debug_tool --compare-output <actual_output> [input_file] [expected_output]
```

默认文件名如下：

- `input_file` 默认为 `data.in`
- `expected_output` 默认为 `data.out`
- `actual_output` 默认为 `actual.out`

第一种模式用于运行学生程序并比对输出。`program_or_cpp` 可以是已经编译好的可执行文件，也可以是 `.cpp` 源文件。如果传入 `.cpp` 源文件，工具会尝试先调用 `clang++ -std=c++17 -O2` 编译，再使用 `data.in` 运行程序。

```
1 ./debug_tool_cli ./student_program data.in data.out actual.out
2 ./debug_tool_cli final_copy_7.cpp data.in data.out
```

第二种模式用于直接分析已有输出文件，适合已经手动运行过学生程序、或只想演示差异定位功能的情况。

```
1 ./debug_tool_cli --compare-output my.out data.in data.out
2 ./debug_tool_cli --compare-output my2.out data.in data.out
```

如果输出完全一致，工具会报告无差异和已检查的总行数；如果发现差异，工具会报告首个错误行，并输出 Expected/Actual、所属 Case、时间、关注城市窗口、bug 前 1 小时事件回放和司令部状态。

Windows 下命令格式相同，只是可执行文件名通常是 `debug_tool.exe`，路径分隔符也可以写成反斜杠：

```
1 .\debug_tool.exe --compare-output my.out data.in data.out
2 .\debug_tool.exe .\student.exe data.in data.out actual_cpp.out
```

Qt 图形界面版 `qt_debug_tool.cpp` 在命令行能力的基础上提供可视化调试界面。界面支持选择源码或可执行文件，也支持直接选择已有输出文件进行比对。分析完成后，界面会展示首个差异摘要、事件回放表、两侧命令部和最多 10 座城市的滑动窗口。测试数据城市数超过 10 时，可以通过滑块切换城市窗口；点击司令部或城市后，右侧会显示该位置相关的武士快照、武器信息、生命值、攻击力和最近事件。若某条事件在标准输出和实际输出中不一致，界面会用红色标记，便于现场演示和答辩。

macOS 下常用命令如下。压缩包中如果已经带有 `debug_tool_cli`，可以直接运行；如果换了机器，也可以重新编译：

```
1 clang++ -std=c++17 -O2 debug_tool.cpp -o debug_tool_cli
2 ./debug_tool_cli --compare-output my.out data.in data.out
3 ./debug_tool_cli some_cpp_file_with_bug/actual.cpp data.in data.out /tmp/moshou_actual_cpp.out
```

macOS 下 Qt 版本可通过 qmake 编译：

```
1 qmake qt_debug_tool.pro
2 make
3 ./qt_debug_tool
```

Windows 和 macOS 的流程不完全一样。Windows 不能直接运行 macOS 下编译出的 `debug_tool_cli` 或 `qt_debug_tool`，需要在 Windows 环境中重新编译生成 `.exe` 文件。推荐使用 MSYS2 MinGW64/Clang64、Qt Creator 或已经配置好 `clang++/g++` 的 PowerShell。

Windows 命令行版推荐流程如下。先在 Windows 上重新编译调试工具：

```
1 clang++ -std=c++17 -O2 debug_tool.cpp -o debug_tool.exe
2 .\debug_tool.exe --compare-output my.out data.in data.out
```

如果 Windows 上没有 `clang++`，也可以使用 MinGW 的 `g++` 编译命令行版：

```
1 g++ -std=c++17 -O2 debug_tool.cpp -o debug_tool.exe
2 .\debug_tool.exe --compare-output my.out data.in data.out
```

Windows 原生 PowerShell/cmd 下，推荐先把学生源码编译成 `.exe`，再把 `.exe` 传给调试工具：

```
1 g++ -std=c++17 -O2 some_cpp_file_with_bug\actual.cpp -o student.exe
2 .\debug_tool.exe .\student.exe data.in data.out actual_cpp.out
```

需要注意：当前工具的“直接传入 `.cpp` 并自动编译运行”功能内部默认调用 `clang++`，并且更适合在 MSYS2 MinGW64/Clang64 这类类 Unix shell 中使用。如果在 MSYS2 环境中已经配置好 `clang++`，也可以这样运行：

```
1 ./debug_tool.exe some_cpp_file_with_bug/actual.cpp data.in data.out actual_cpp.out
```

Windows 下 Qt 图形界面版可以用 Qt Creator 打开 `qt_debug_tool.pro` 后直接构建运行；也可以在 Qt 对应的命令行环境中执行：

```
1 qmake qt_debug_tool.pro
2 mingw32-make
3 .\release\qt_debug_tool.exe
```

如果 Qt 使用的是 Debug 构建，生成文件可能位于 `.\debug\qt_debug_tool.exe`。打开界面后，使用方式与 macOS 相同：选择“运行学生程序/源码”或“比对已有输出”，再选择 `data.in`、`data.out` 和实际输出文件。

2. 项目各模块与类设计细节

项目主要由命令行调试模块、输出解析与差异分析模块、回放筛选模块、Qt 可视化模块和样例数据模块组成。

2.1 命令行调试模块

命令行版入口位于 `debug_tool.cpp` 的 `main` 函数。它首先解析命令行参数，判断当前是运行程序并比对，还是直接比对已有输出。如果输入是 `.cpp` 文件，`runTarget` 会先生成临时可执行文件，再使用输入重定向运行目标程序，并把输出保存到指定路径。这样设计后，使用者不需要手动编译学生源码，也能直接把有 bug 的 C++ 作业文件交给调试工具分析。

相关函数包括：

- `endsWith`：判断输入路径是否是 `.cpp` 源文件。
- `shellQuote`：对路径做 shell 引号处理，降低路径中特殊字符带来的运行风险。
- `runTarget`：负责编译源码、运行程序、重定向输入输出。
- `printUsage`：输出命令行使用说明。

2.2 输出解析与差异分析模块

本工具没有重新实现完整的魔兽世界战斗模拟器，而是基于标准输出文本进行事件级分析。这样做的优点是适配性更强，不依赖学生代码内部结构，只要学生程序最终输出符合题目格式，就可以直接分析。

核心数据结构包括：

- `CaseConfig`：保存每个测试用例的基础配置，例如生命元、城市数、箭攻击力、忠诚度下降值和结束时间。当前主要使用城市数 `N` 来计算可视化窗口和关注区域。
- `LineInfo`：为输出中的每一行补充结构化信息，包括全局行号、所属 Case、Case 内部行号、时间戳、相关城市、是否关联红/蓝司令部以及原始文本。
- `CompareResult`：保存逐行比对结果，包括是否完全一致、首个差异行下标、标准输出内容、实际输出内容，以及是否存在某一方提前结束的情况。

主要函数包括：

- `readCases`：读取 `data.in` 中每个测试用例的头部配置。
- `readLines`：读取标准输出或实际输出，并兼容 Windows 的 CRLF 换行。
- `parseMinute`：将 `004:10` 形式的时间戳转换为分钟数，便于计算前 1 小时窗口。
- `extractCities`：从输出行中提取 `city x` 信息。
- `buildLineInfo`：把纯文本输出转换成带时间、Case、城市和司令部标签的结构化行。
- `compareLines`：逐行比较标准输出和实际输出，定位首个不同位置。

2.3 回放筛选模块

回放模块负责把“第一处差异”扩展成一段可读的上下文。工具会根据差异行推断关注区域：如果差异行出现了具体城市，则以该城市为中心展示最多 10 个城市；如果差异发生在红司令部或蓝司令部，则窗口贴近对应司令部；如果无法判断具体位置，则回退到当前 Case 的前 10 个城市。

相关函数包括：

- `addCityWindow`：根据起点和城市总数生成最多 10 个城市的窗口。
- `buildFocusCities`：根据首个差异行推断关注城市集合。
- `touchesFocus`：判断某条历史事件是否与关注区域相关。
- `reportReplayWindow`：输出 bug 前 1 小时的相关事件。
- `reportHeadquarterStatus`：提取差异发生前最近一次红蓝司令部生命元状态。
- `printExcerpt`：输出差异点附近的标准输出和实际输出片段。

这种实现没有重建完整战局状态，而是从标准答案中筛选关键事件。它更适合作为 debug 定位工具：先帮助使用者找到“第一个错在哪里”，再围绕该时间点和区域回看武士移动、战斗、司令部生命元和占领事件。

2.4 Qt 可视化模块

Qt 版本复用了命令行版的核心分析思路，并额外增加了图形界面、事件表和位置详情面板。主要结构包括：

- `ReplayEvent`：保存回放窗口内的一条事件，同时记录实际输出对应行和是否存在差异。
- `WarriorSnapshot`：从事件文本中提取武士阵营、类型、编号、城市、生命值、攻击力和武器信息，用于右侧位置详情展示。
- `AnalysisReport`：Qt 分析流程的总结对象，包含差异结果、输入配置、标准/实际输出、结构化行信息、回放事件、武士快照、关注城市和司令部状态。
- `LocationItem`：继承自 `QGraphicsObject`，表示地图中的一个司令部或城市节点。它负责绘制节点标题、摘要信息和差异警示边框，并在点击时发出选中信号。
- `MainWindow`：主窗口类，负责构建界面、处理文件选择、编译/运行学生程序、调用分析函数、刷新摘要、地图、事件表和位置详情。

`MainWindow` 中的界面被分成几个区域：

1. 顶部输入区：选择运行模式、学生程序或源码、输入文件、标准输出和实际输出。
2. 左上摘要区：展示首个差异、Case、时间、Expected/Actual、回放窗口和司令部状态。
3. 中部地图区：展示红司令部、蓝司令部和最多 10 个城市，并支持滑块切换城市窗口。
4. 下方事件表：列出回放窗口内的标准事件和实际事件，差异行用红色标记。
5. 右侧详情区：点击城市或司令部后，展示相关武士快照和该位置的事件文本。

Qt 版中与分析有关的函数包括 `analyzeFiles`、`buildSnapshots`、`lastHeadquarterStatus`、`populateReport`、`populateMap`、`populateEvents` 和 `populateDetail`。其中 `analyzeFiles` 是图形界面的分析核心，它会读取文件、比对输出、生成关注城市、筛选事件、构造武士快照，并最终返回 `AnalysisReport` 给界面渲染。

2.5 样例数据与提交文件

提交目录中包含以下关键文件：

- `debug_tool.cpp`：命令行版源码。
- `debug_tool_cli`：当前 macOS 环境下已编译的命令行版可执行文件；Windows 下需要重新编译生成 `debug_tool.exe`。
- `qt_debug_tool.cpp`：Qt 图形界面版源码。
- `qt_debug_tool.pro`：Qt/qmake 工程文件。
- `data.in`：测试输入。
- `data.out`：标准输出。
- `my.out`：带首个差异的样例输出，可演示 bug 定位。
- `my2.out`、`my3.out`：无差异样例输出，可演示正确输出识别。
- `some_cpp_file_with_bug/`：用于演示直接传入 `.cpp` 源码并自动编译运行的样例文件。
- `魔兽世界终极版.md`：题目规则说明。
- `魔兽作业debug工具功能设计文档.md`：更详细的功能设计文档。

3. 项目总结与反思

本项目完成了从输出比对到局部回放再到图形化展示的基本闭环。命令行版适合快速验证和批量调试，Qt 版适合现场演示和更直观地观察错误发生前后的局部战场状态。相比人工逐行查找，工具可以直接定位第一处差异，并把关注范围收敛到 bug 前 1 小时和最多 10 个城市，大幅减少排查信息量。

项目中比较实用的设计是没有强行重写一套完整战斗模拟器，而是先利用标准输出文本建立轻量事件索引。这样实现成本较低，也更容易适配不同学生代码。只要标准输出和实际输出存在首个差异，工具就能根据题目固定输出格式提取时间、城市和司令部信息，进而生成可读的回放。

当前版本仍有一些不足。第一，回放是基于输出文本筛选的，并不等同于完整的战斗状态重建，因此某些隐藏状态变化只能通过已输出事件间接观察。第二，错误原因目前主要交给使用者判断，工具还没有根据差异类型自动给出“生命元统计时机错误”“司令部占领顺序错误”等更具体的诊断。第三，源码运行依赖本机存在 `clang++`，Qt 版也依赖 Qt/qmake 环境，跨平台打包还可以继续完善。

后续如果继续扩展，可以从三个方向改进：一是增加更细的错误类型识别规则，让工具不仅指出第一处差异，还能给出可能原因；二是逐步重建部分战斗状态，例如武士生命值、武器、城市旗帜和司令部生命元，使界面展示更接近真实战局；三是把分析结果导出为 JSON，方便后续接入更多前端界面或自动化测试流程。